

Engenharia de Software

Faculdade Jorge Amado
Prof. David Viscarra

Padrões de Projeto (Design Patterns)

Engenharia de requisitos...

Nesta aula, abordaremos Padrões de Projeto (Design Patterns):

- **Definição**
- **Objetivo**
- **Classificação**
- **Vantagens**
- **Desvantagens**
- **Quando usa**
- **Casos de uso – Exemplos práticos**

Padrões de Projeto (Design Patterns)...

Início

- Os padrões de projeto surgiram de um conceito criado pelo arquiteto Christopher Alexander,
- Em sua obra *"The Timeless Way of Building"*, de 1979
- Propôs que cada padrão é uma regra com três partes, que expressa a relação entre:
 - contexto,
 - problema e
 - solução

Padrões de Projeto (Design Patterns)...

A essência da documentação de um padrão:

- O **Contexto**: se refere ao conjunto de situações que se repetem, nas quais o padrão de projeto pode ser aplicado;
- O **Problema**: que trata do conjunto de “forças” – objetivos e limitações – que ocorrem dentro do contexto;
- A **Solução**: que é uma estrutura formal para ser aplicada na resolução do problema

🤝 A primeira iniciativa em aplicar padrões no desenvolvimento de software veio 1995;

🤝 três colaboradores (Helm, Johnson e Vlissides)

🤝 Obra intitulado “*Design Patterns: Elements of Reusable Object-Oriented Software*”

🤝 autores identificaram 23 padrões para projetos orientados a objetos

Padrões de Projeto (Design Patterns)...

Objetivos

Os principais objetivos dos padrões de projeto são:

- **Reutilização de soluções eficazes**, evitando "reinventar a roda".
- **Facilitar a manutenção e evolução** do software.
- **Melhorar a comunicação** entre desenvolvedores usando uma linguagem comum.
- **Promover a separação de responsabilidades** e a aderência a princípios SOLID.
- **Reduzir o acoplamento** entre componentes do sistema.

Classificação dos Padrões...

Os padrões são geralmente divididos em três categorias:

- **Padrões Criacionais:**
Lidam com a criação de objetos, abstraindo o processo de instanciamento.
Exemplos: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
- **Padrões Estruturais:**
Ajudam a compor classes e objetos em estruturas maiores, promovendo reutilização.
Exemplos: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
- **Padrões Comportamentais:**
Focam na comunicação entre objetos, delegação de responsabilidades e fluxos de controle.
Exemplos: Observer, Strategy, Command, State, Iterator, Template Method, Mediator.



Vantagens dos Padrões de Projeto...

-  **Reutilização de soluções consolidadas:** Evita erros comuns.
-  **Documentação e aprendizado facilitado:** Muitos desenvolvedores já conhecem esses padrões.
-  **Facilidade de manutenção e extensão:** Sistemas bem estruturados são mais fáceis de modificar.
-  **Melhora na comunicação da equipe:** Usar nomes como "*Observer*", "*Factory*", etc., torna a comunicação mais eficiente.
-  **Aderência a princípios de design:** Incentivam modularidade, coesão e baixo acoplamento.



Desvantagens dos Padrões de Projeto...

-  **Curva de aprendizado:** Exige conhecimento prévio e entendimento do problema para aplicar corretamente.
-  **Aplicações excessivas (overengineering):** Usar padrões sem necessidade pode tornar o sistema complexo.
-  **Aumento da abstração:** Alguns padrões introduzem níveis extras de abstração que podem dificultar a compreensão.
-  **Código mais verboso:** A implementação pode exigir mais classes e interfaces, aumentando a complexidade inicial.

Quando Usar Padrões de Projeto...

- Quando há **problemas recorrentes** no design de software.
- Quando se deseja **flexibilidade e extensibilidade** futuras.
- Quando a equipe precisa **padronizar soluções** e facilitar a colaboração.
- Quando se pretende **seguir boas práticas** de arquitetura e engenharia de software.

Exemplos Práticos...

- Um sistema que precisa executar diferentes estratégias de ordenação → **Strategy**
- Um sistema que precisa registrar e notificar mudanças em tempo real → **Observer**
- Um sistema que precisa instanciar objetos com base em parâmetros → **Factory Method**
- Um sistema que deseja acessar objetos remotos ou pesados de forma controlada → **Proxy**

Factory Method – Criacional Exemplo (Java)

```
• // Produto
• interface Transporte {
•     void entregar();
• }

• // Produtos concretos
• class Caminhao implements Transporte {
•     public void entregar() {
•         System.out.println("Entrega por caminhão");
•     }
• }

• class Navio implements Transporte {
•     public void entregar() {
•         System.out.println("Entrega por navio");
•     }
• }

• // Criador
• abstract class Logistica {
•     abstract Transporte criarTransporte();
• }

• // Criadores concretos
• class LogisticaRodoviaria extends Logistica {
•     public Transporte criarTransporte() {
•         return new Caminhao();
•     }
• }

• class LogisticaMaritima extends Logistica {
•     public Transporte criarTransporte() {
•         return new Navio();
•     }
• }
```

Factory Method – Exemplo (Java)

- ```
interface Transporte {
 void entregar();
}

class Caminhao implements Transporte {
 public void entregar() {
 System.out.println("Entrega por caminhão");
 }
}
```

# Adapter – Exemplo (Java)

- `// Interface esperada`
- `interface LeitorPDF {`
- `void ler(String arquivo);`
- `}`
  
- `// Classe existente incompatível`
- `class LeitorLegacy {`
- `void abrirArquivo(String nome) {`
- `System.out.println("Abrindo arquivo: " + nome);`
- `}`
- `}`
  
- `// Adaptador`
- `class AdaptadorLeitorPDF implements LeitorPDF {`
- `private LeitorLegacy legacy = new LeitorLegacy();`
  
- `public void ler(String arquivo) {`
- `legacy.abrirArquivo(arquivo);`
- `}`
- `}`

# Adapter – Exemplo (Java)

- ```
interface LeitorPDF {  
    void ler(String arquivo);  
}
```



```
class LeitorLegacy {  
    void abrirArquivo(String nome) {  
        System.out.println("Abrindo: " + nome);  
    }  
}
```

Command – Exemplo (Java)

```
• // Comando
• interface Comando {
•     void executar();
• }

• // Receptor
• class Luz {
•     void ligar() { System.out.println("Luz ligada"); }
•     void desligar() { System.out.println("Luz desligada"); }
• }

• // Comandos concretos
• class LigarLuz implements Comando {
•     private Luz luz;

•     LigarLuz(Luz luz) { this.luz = luz; }

•     public void executar() { luz.ligar(); }
• }

• class DesligarLuz implements Comando {
•     private Luz luz;

•     DesligarLuz(Luz luz) { this.luz = luz; }

•     public void executar() { luz.desligar(); }
• }

• // Invocador
• class ControleRemoto {
•     void acionar(Comando comando) {
•         comando.executar();
•     }
• }
```

Command – Exemplo (Java)

- ```
interface Comando {
 void executar();
}

class Luz {
 void ligar() { System.out.println("Luz ligada"); }
}
```



# Strategy – Exemplo (Java)

```
• // Estratégia
• interface EstrategiaOrdenacao {
• void ordenar(int[] numeros);
• }

• // Estratégias concretas
• class OrdenacaoCrescente implements EstrategiaOrdenacao {
• public void ordenar(int[] numeros) {
• Arrays.sort(numeros);
• System.out.println("Ordenado crescente: " + Arrays.toString(numeros));
• }
• }

• class OrdenacaoDecrescente implements EstrategiaOrdenacao {
• public void ordenar(int[] numeros) {
• Arrays.sort(numeros);
• for (int i = 0; i < numeros.length / 2; i++) {
• int temp = numeros[i];
• numeros[i] = numeros[numeros.length - 1 - i];
• numeros[numeros.length - 1 - i] = temp;
• }
• System.out.println("Ordenado decrescente: " + Arrays.toString(numeros));
• }
• }

• // Contexto
• class Ordenador {
• private EstrategiaOrdenacao estrategia;

• Ordenador(EstrategiaOrdenacao estrategia) {
• this.estrategia = estrategia;
• }

• void ordenar(int[] numeros) {
• estrategia.ordenar(numeros);
• }
• }
```

# Strategy – Exemplo (Java)

- ```
interface EstrategiaOrdenacao {  
    void ordenar(int[] numeros);  
}  
  
class OrdenacaoCrescente implements EstrategiaOrdenacao {  
    public void ordenar(int[] n) {  
        Arrays.sort(n);  
    }  
}
```

Conclusões...

- Padrões ajudam a criar **sistemas robustos e bem projetados**
- São ferramentas poderosas, **mas exigem critério no uso**
- Estudar padrões é essencial para se tornar um **engenheiro de software completo**

Dicas:

-  **Leia o livro “Design Patterns” da GoF (Gang of Four)**
-  **Estude os diagramas de cada padrão**
-  **Implemente exemplos simples em código**

Bibliografia

- **“Agile Principles, Patterns and Practices in C#”** é um livro de mais de 700 páginas, publicado em 2006 e escrito por Robert C. Martin (vulgo, “Uncle Bob”) e seu filho Micah Martin.
- DAVIS, A. M. Software Requirements: objects, functions and states. Englewood Cliffs, New Jersey: Prentice Hall. 1993.
- KOTONYA, G.; SOMMERVILLE, I. Requirements Engineering: processes and techniques (worldwide series in computer science). Wiley, 1998.
- PRESSMAN, Roger S. PENTEADO, Rosângela Delloso (Trad.). Engenharia de software. 6. ed. Rio de Janeiro: McGrawHill, 2006.
- PRESSMAN, Roger S. Engenharia de software: Uma abordagem Profissional. 7. ed. São Paulo: McGraw-Hill, 2011.

